SYNTHGRAPH | ENGINEERING NOTEBOOK

Experiment Workspace

Performance Issues, Treated as Experiments

Problem (X) → Solution (Y) → Result (Z): expected versus actual

Item	Detail
Subject	Slow tab navigation, repeated RSC requests and oversized payloads on the experiment detail page
Measured on	30 October 2026, production (`synthgraph.dev`), test account, read-only requests
Experiment used	Phase 1, Domain-Randomized Quadruped Locomotion: 16 generations, 3 training runs, about 60 metric steps per run
Code state	Before: `main` at commit `9e32854`. After: commits `379c83c` (tab clicks, rate limiting) and `1b1eb67` (one workspace request, timing), both deployed and re-measured
Stack	Next.js 16, React 19 (frontend); NestJS (backend); hosted on Vercel
Status	**Implemented in two deploys (379c83c, 1b1eb67) and re-measured on production on 30 October 2026. Tab clicks: fixed. Full page load: about 2.3 times faster (server render 2015 to 871 ms). PERF-02 largely done; PERF-05 partly done.**

How to use this notebook

Write the **expected** result first. Measure afterwards with the same method. Then fill in the **actual** result and note any gap: the gap is where the learning is.

Where a number is inferred rather than measured, the text says so.

Contents

1. Executive summary 3
2. How to read this notebook 5
3. Measurement method and baseline 6
4. The issues 7
PERF-01 Every tab or inspector click is a server navigation 7
PERF-02 One page load makes about 33 backend calls in 3 sequential hops 11
PERF-03 Normal browsing exhausts the backend rate limit 14
PERF-04 Failed manifest loads are swallowed and look like "no data" 16
PERF-05 The page props carry duplicated and unneeded data 18
PERF-06 Revisiting a tab reuses nothing 20
PERF-07 The workspace cannot tell us where its time goes 22
PERF-08 The tab animation adds a fixed wait 24
Suggested order of work 25
Open questions 25
Results log 25

1. Executive summary

The bottom line

Every tab click makes a full server round trip that re-fetches the whole workspace, even though the browser already holds all of that data. The tab does not change until the server finishes, which takes **1.0 to 2.1 seconds**.

Each round trip costs about **33 backend calls**. That volume also trips the backend rate limit after roughly 4 clicks a minute, after which reproduction manifests silently fail and the Reproduction tab shows wrong data.

The Python source repeated in the payload is wasteful, but it is **not** the cause of the delay.

The outcome, measured on production after the fix

A tab click now changes the screen in **8 to 24 ms** (median 13 ms) with **0 server requests**, down from 1387 to 2601 ms (median 1922 ms). Thirty clicks in a burst made no requests and left the Reproduction tab with 16 of 16 manifests loaded.

A **full page load is now faster too**, after a second change that loads the whole workspace in one backend request: the server render fell from **2015 ms to a median of 871 ms**, and Refresh data from 2.1 s to 1.3 s. After the first deploy alone it was still about 2.0 s, which confirmed that the three sequential hops, not the call count, were the cost. A hard page load is about 1.6 s (1.1 to 2.0 s). PERF-05 is only partly done: the payload is 32 percent smaller (403 to 274 KB) but the Python source is still sent twice.

Not exercised on production: the rate-limit failure path (nothing was throttled) and the per-user throttle (that would be a stress test of the live site). Both were verified on a local build.

The eight issues at a glance

ID	Issue	Class	Impact	Effort
PERF-01	Every tab or inspector click is a server navigation	**Architecture**	**High**	Small
PERF-02	About 33 backend calls in 3 sequential hops	**System design**	**High**	Medium
PERF-03	Normal browsing exhausts the backend rate limit	**System design / Reliability**	**High**	Small (backend)
PERF-04	Failed manifest loads look like "no data"	**Reliability**	**High**	Small
PERF-05	Page props carry duplicated and unneeded data	**Data design**	**Low-Medium**	Small-Medium
PERF-06	Revisiting a tab reuses nothing	**Caching and state**	**High**	None beyond PERF-01
PERF-07	No visibility into where the time goes	**Observability**	**Medium**	Small
PERF-08	Tab animation adds a fixed wait	**Frontend rendering**	**Low**	Trivial

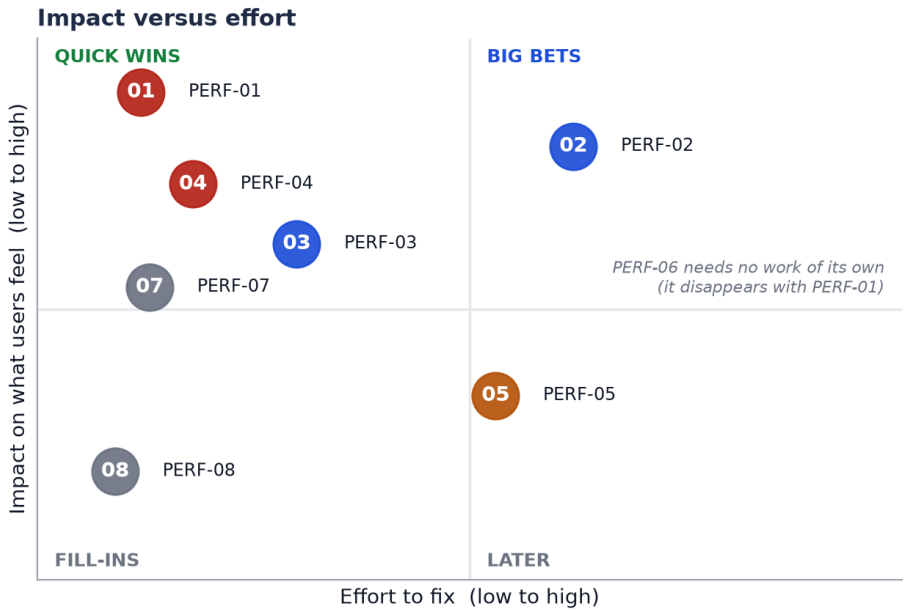


Figure 0. Impact versus effort. PERF-01 and PERF-04 are the quick wins; PERF-02 is the larger bet.

2. How to read this notebook

Each issue is a small lab experiment with three parts, colour-coded so you can find them quickly.

Part	Meaning	Where the evidence comes from
X The problem	What went wrong, where in the code, and how we know. Includes the mechanism, not just the symptom.	Measurements taken on production, plus the actual source files.
Y The solution	The proposed fix, why it beats the alternatives, and what we rejected on purpose.	Reasoning against the code and the Next.js 16 documentation bundled with the project.
Z The result	A table with the baseline, the expected result (hypothesis) and the actual result measured on production after the fix.	Re-running the same measurement after the change.

Classification

Each issue is tagged with one class, so you can see which kind of problem it is and which layer owns the fix.

Class	What it covers	The question it answers
Architecture	How the frontend is structured: routing, the server/client component boundary, where state lives.	Is the work happening in the right layer?
System design	How the frontend, its server layer and the backend cooperate: API granularity, fan-out, rate limits.	Is the whole system shaped sensibly under real load?
Reliability	What happens when something fails or is limited.	Does it fail loudly and correctly?
Data design	What crosses the server-to-client boundary, and in what shape.	Are we sending the right bytes?
Caching and state	What is reused, for how long, and who may see it.	Why did it fetch that again?
Observability	Whether we can see what the system is doing.	How would we know?
Frontend rendering	Client-side rendering and animation cost.	Is the browser doing needless work?

3. Measurement method and baseline

How to measure

Open the experiment page signed in, then paste this in the DevTools console. For each tab it reports how long the UI took to change and which network request happened. Use the same script before and after every change, run it at least three times, and treat differences under about 20% as noise.

```
performance.clearResourceTimings();
const clickTab = (label) => new Promise((resolve) => {
  const tabs = () => [...document.querySelectorAll('[role=tablist] button')];
  const button = tabs().find((b) => b.innerText.trim() === label);
  const start = performance.now();
  const seen = performance.getEntriesByType('resource').length;
  button.click();
  const timer = setInterval(() => {
    const sel = tabs().find((b) => b.getAttribute('aria-selected') === 'true')?.innerText.trim();
    if (new URLSearchParams(location.search).get('tab') === label.toLowerCase() && sel === label) {
      clearInterval(timer);
      const rsc = performance.getEntriesByType('resource').slice(seen).filter((r) =>
r.name.includes('_rsc'));
      resolve({ label, uiMs: Math.round(performance.now() - start), rscRequests: rsc.length,
        rscMs: rsc.map((r) => Math.round(r.duration)), decodedKB: rsc.map((r) =>
Math.round(r.decodedBodySize / 1024)) });
    }
  }, 20);
  setTimeout(() => { clearInterval(timer); resolve({ label, timeout: true }); }, 15000);
});
const results = [];
for (const l of ['Lineage','Runs','Metrics','Lineage','Parameters','Reproduction','Overview','Lineage']) {
  results.push(await clickTab(l));
  await new Promise((r) => setTimeout(r, 600));
}
console.table(results);
```

For the reliability checks, also note how many generations come back without a manifest after 10 rapid clicks (compare the Reproduction tab's "N partly reproducible" count with the generation count).

Baseline: eight tab clicks on production

#	Click	UI changed after	RSC calls	Request	TTFB	Body wait	Decoded
1	Lineage	2060 ms	1	2030 ms	160 ms	1858 ms	400 KB
2	Runs	1542 ms	1	1501 ms	161 ms	1338 ms	400 KB
3	Metrics	2111 ms	1	2089 ms	193 ms	1894 ms	400 KB
4	Lineage (revisit)	1426 ms	1	1389 ms	189 ms	1197 ms	400 KB
5	Parameters	1450 ms	1	1411 ms	169 ms	1239 ms	400 KB
6	Reproduction	1516 ms	1	1485 ms	213 ms	1270 ms	400 KB
7	Overview	2134 ms	1	2117 ms	171 ms	1943 ms	335 KB
8	Lineage (revisit)	1048 ms	1	1035 ms	170 ms	863 ms	262 KB

Compressed, every response was about 25 KB. Rows 7 and 8 are smaller because rate limiting had started (PERF-03).

4. The issues

Each issue starts on its own page and follows the same X, Y, Z structure.

PERF-01 Every tab or inspector click is a server navigation

CLASSIFICATION	IMPACT	FIX EFFORT	CONFIDENCE
Architecture	High	Small	High

X The problem encountered

What happens in the code

The six tabs are `?tab=` query parameters. Both `shared/ui/Tabs.tsx` (`selectTab`) and `ExperimentWorkspace.tsx` (`setTab`, `setSelected`) call `router.push("?...")`. In the App Router, a navigation to a new search string re-runs the server page. `experiments/[experimentId]/page.tsx` ignores the tab, loads everything, and hands it to `ExperimentWorkspace`, a client component.

Why that is wasteful

Every tab body (`OverviewTab`, `RunsTab`, `LineageTab`, `MetricsTab`, `ParametersTab`, `ReproductionTab`) is a pure function of props the browser already holds. A tab click only changes which of those props gets drawn. The server response contains no new information.

Why it feels slow

`activeTab` is read from `useSearchParams()`, and the URL only commits after the server responds. The UI therefore sits on the old tab for the whole request. Measured: the UI changed within about 40 ms of the request finishing, every time.

> **Measured on production**
>
> 8 clicks produced 8 `_rsc` requests, 1.0 to 2.1 s each. Revisits were no faster than first visits.
>
> Entity selection (clicking a run or generation) uses the same `router.push`, so it has the same cost. It was not timed separately.
>
> It is not slow transfer (25 KB compressed) and not slow client rendering (about 40 ms between response and UI change).

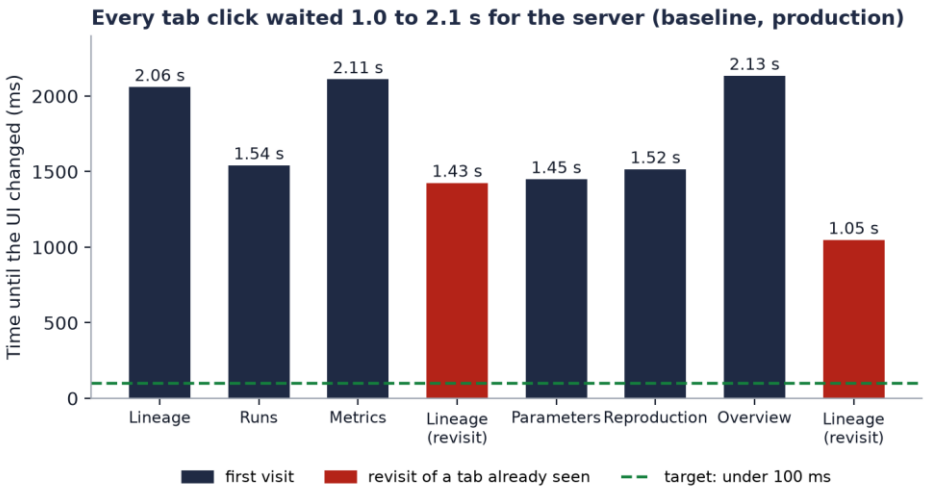


Figure 1. Time until the UI changed, per click. Red bars are revisits.

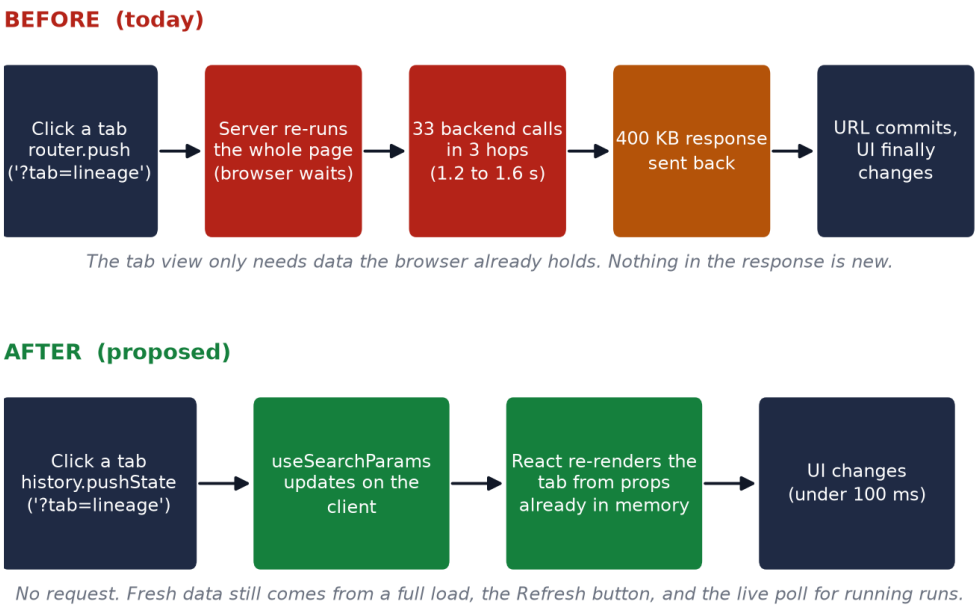


Figure 2. The navigation path today versus the proposed one.

Y The solution and why it is the most optimised

Update the URL without asking the server. Replace `router.push` with `window.history.pushState` for `?tab=` and `?entity=`. Next.js documents this pattern (`01-app/02-guides/single-page-applications.md`) and keeps `useSearchParams` and `usePathname` in sync with it.

Practical shape

- Add an opt-in prop to the shared `Tabs` component (for example `shallow`), so other pages that rely on server-driven tabs are unaffected.
- Make the same change in `ExperimentWorkspace.setTab` and `setSelected`.
- Add a visible **Refresh** action that calls `router.refresh()`. Running runs already poll every 5 seconds through `useLiveTrainingRun`.

Why this is the most optimised fix

1. It removes the cause instead of speeding up the symptom. The fastest request is the one never made.

2. The URL stays the source of truth, so deep links, bookmarks, back and forward all keep working. That was the original design goal (see the `Tabs.tsx` comment).
3. It is a small change in two files with no new dependency and no routing rewrite.
4. It adds no caching, so nothing can go stale and there is no cross-user risk. The data in memory is what the server already sent to this user's session.

Alternatives considered and rejected

Alternative	Why not
`staleTimes.dynamic` **(router cache)**	Experimental. Applies to every dynamic page, so it would show stale data everywhere. First visits would still be slow.
A separate route per tab	Still re-fetches per route unless the data moves into a shared layout. A large rewrite for no extra benefit.
Tabs as local React state only	Loses shareable URLs and the browser back button.
Prefetching each tab	Every tab link would prefetch the same 400 KB again, because the page ignores its tab.

> **Trade-off to accept**
> A tab click will no longer refresh the data as an accidental side effect. That is why the Refresh action is part of the solution.

Z The result: expected versus actual

The Expected column is the hypothesis, written before any code. The Actual column was filled in on 30 October 2026 by re-running the same method on production after each deploy (commits `379c83c` and `1b1eb67`). Anything not re-tested on production says so.

Measure	Baseline (measured)	Expected result (hypothesis)	Actual result (production, after deploy)
`_rsc` requests per tab click	1	0	0 on all 8 clicks, and 0 new requests during a 30-click burst 40 ms apart. Met.
UI change after a click	1.0 to 2.1 s	under 100 ms (one React render, plus the 200 ms enter animation if kept)	8 to 24 ms, median 13 ms (was 1387 to 2601 ms, median 1922 ms). Met, about 150 times faster.
Backend calls per tab click	about 33	0	0 (no request leaves the browser, so none reaches the backend). Met.
Revisit versus first visit	same slow time	both fast	Lineage: 8 ms first visit, 14 ms and 12 ms on revisits. Met.
Back button returns to the previous tab	works	still works	Overview, click Metrics, back gives Overview, forward gives Metrics; URL followed each time. Met.
Deep link `?tab=lineage` on a fresh load	works	still works	A fresh load of `?tab=metrics` opened the Metrics tab. Met.

What to learn from the gap

The gap was in our favour: the expectation was under 100 ms and the slowest tab took 24 ms (Runs), so client render cost is not hiding behind the old wait. The visible cost that remains is the full page load (about 2 s), which PERF-02 covers.

PERF-02 One page load makes about 33 backend calls in 3 sequential hops

CLASSIFICATION	IMPACT	FIX EFFORT	CONFIDENCE
System design	High	Medium	Medium-high

X The problem encountered

What happens in the code

`page.tsx` fetches in this order:

1. `requireSession()` calls `/auth/me/api-key`. One call.
2. In parallel: the experiment, the parameter-correlation report, the runs list and the generations list. Four calls.
3. For every run: metrics, evaluations, health and drift. For every generation: its reproduction manifest.

The total is 5 + 4N + G calls, where N is the number of runs and G the number of generations. For this experiment that is 5 + 12 + 16 = 33 calls in 3 dependent hops, because step 3 needs the lists from step 2, and step 2 needs the session from step 1.

Why it is slow

The browser sees a quick shell (TTFB 160 to 210 ms) and then waits. The stream barely moves until about 1.6 s, then delivers the data in one burst. As a proxy for backend latency, `/api/training-runs/:id/live` makes 3 parallel backend calls and took 545 to 960 ms, so one backend round trip costs roughly 0.4 to 0.5 s. Three sequential hops at that cost land at the observed 1.2 to 1.6 s.

Why it will get worse

The call count scales with experiment size. An experiment with 50 runs and 100 generations would make about 305 calls per page load (computed, not measured).

> **Part of this work is never shown**
> `health` and `drift` are fetched for every run, but they are only displayed in the run inspector's Health tab, not on any tab canvas.

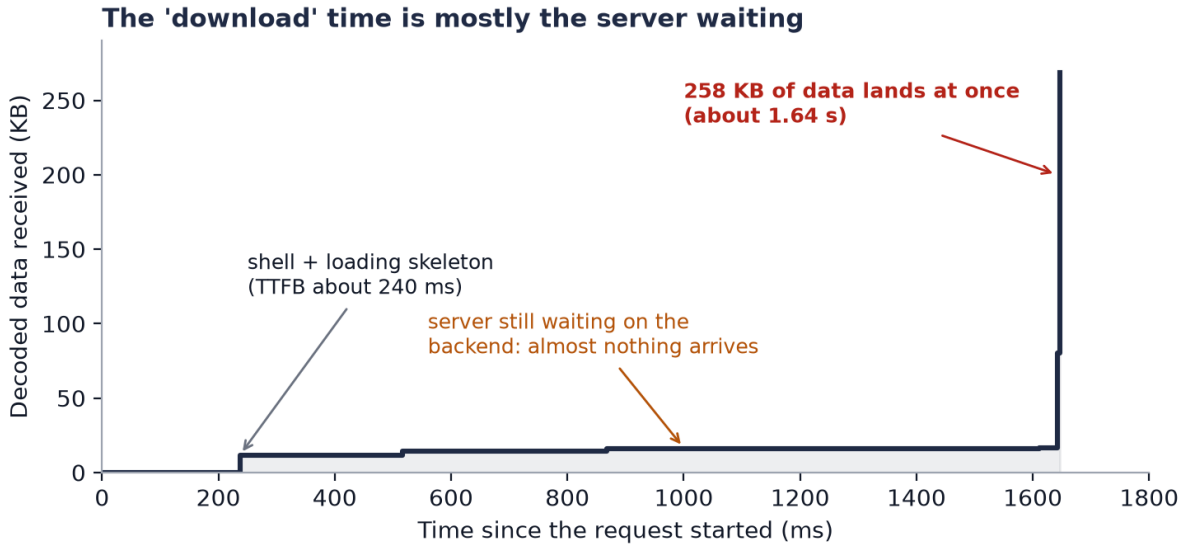


Figure 3. One Lineage response, chunk by chunk. The data arrives in a single burst.

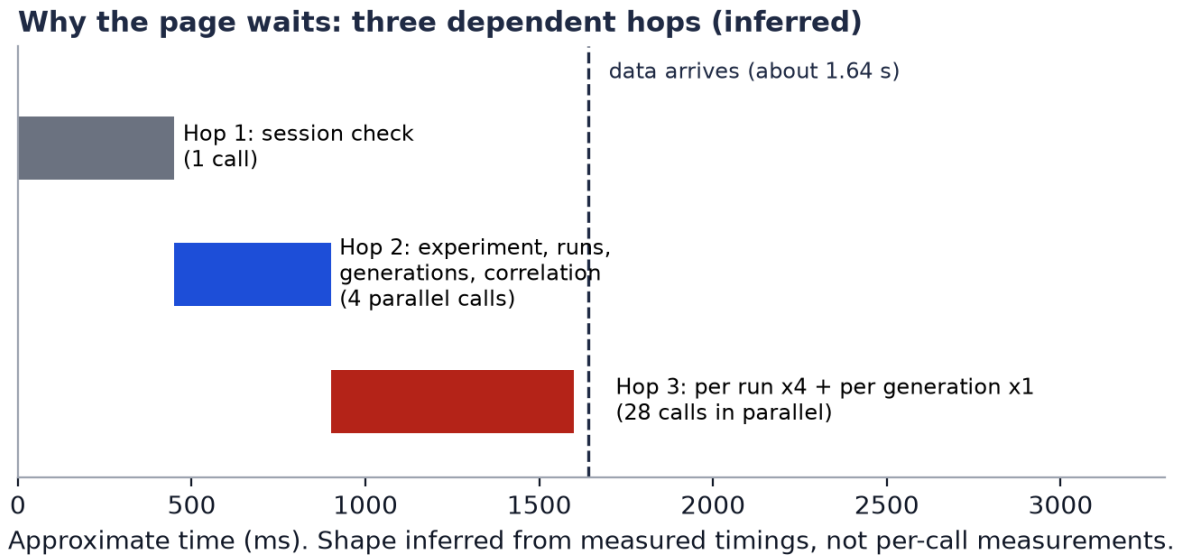


Figure 4. The inferred shape of the three hops. The timing needed to confirm it exists now (PERF-07) but has not been read from production yet.

Y The solution and why it is the most optimised

In order of value:

1. **Load per-run health and drift only when a run is selected in the inspector.** Removes 2N calls from the first load. Frontend only.
2. **Add an additive backend endpoint that returns the workspace in one call** (for example `GET /experiments/:id/workspace`), assembled with database joins. This collapses three hops of many calls into one hop of one call, and follows the additive-only API policy.
3. **Only if the backend struggles under parallel load:** bound the concurrency of whatever fan-out remains.

Why this is the most optimised fix

1. The cost is dominated by the number of sequential network hops, not by payload size, so the lever is fewer hops and fewer calls.
2. Step 1 is the cheapest win and needs no backend change.
3. Step 2 moves joining to the database, where it belongs, instead of many HTTP calls from the server layer. It is also what makes the page scale with experiment size.
4. None of it relies on caching, so freshness and user isolation are unchanged.

Alternatives considered and rejected

Alternative	Why not
Cache backend responses across requests	The data is per-user and changes while runs train. A cache adds staleness and a cross-user leak risk.
Raise parallelism	Each hop is already parallel. The hops themselves are sequential by data dependency.
Fetch per tab on the client (TanStack Query)	Possible later (`QueryProvider` exists), but it moves the same N calls into the browser. It is not smaller.

Z The result: expected versus actual

The Expected column is the hypothesis, written before any code. The Actual column was filled in on 30 October 2026 by re-running the same method on production after each deploy (commits `379c83c` and `1b1eb67`). Anything not re-tested on production says so.

Measure	Baseline (measured)	Expected result (hypothesis)	Actual result (production, after deploy)
Backend calls per full page load	about 33	step 1: about 27. step 2: 1 to 3	First deploy: 18. Second deploy: 3 (the session check, one workspace request, and the page-title lookup alongside), counted on a real backend with a local proxy; not visible from the browser on production. Met.
Sequential hops	3	step 2: 1	2 (the session check, then the workspace request), measured the same way. The target was 1; the session check still runs first. Mostly met.
Time to first data (full load)	about 1.6 s	step 1: about 1.6 s (hops unchanged). step 2: about 0.5 to 0.8 s	First deploy: about 2.0 s, unchanged. Second deploy: server render 834 to 1073 ms, median 871 ms (5 fetches); hard page load 1117 to 1994 ms, median 1604 ms (3 loads). The expected 0.5 to 0.8 s was not quite reached. Mostly met.
Calls with 50 runs and 100 generations	about 305 (computed)	step 2: still 1 to 3	3 requests whatever the size (computed from the design: one workspace request). The backend still does a few queries per run inside it, so its work grows with size. Met for calls.

What to learn from the gap

The note predicted this, and production confirmed it in two steps. Cutting the calls from 33 to 18 in the same three hops changed nothing (about 2.0 s). Removing a hop with the single workspace request brought the server render to 871 ms. So the cost is round trips, not call count or payload. Measured on a real backend with a 150 ms simulated round trip, the page load went from about 660 ms (3 hops, 18 calls) to about 380 ms (2 hops, 3 calls). What is left is the session check running ahead of the data request; running them together would save about another round trip.

PERF-03 Normal browsing exhausts the backend rate limit

CLASSIFICATION	IMPACT	FIX EFFORT	CONFIDENCE
System design / Reliability	High	Small (backend)	High (mechanism)

X The problem encountered

What happens in the code

The backend uses `@nestjs/throttler` with a global default of 120 requests per 60 seconds (`synthgraph-backend/src/app.module.ts:35`) and no custom tracker, so the key is the request IP. A frontend server call comes from the hosting provider's address, not from the user's browser.

Why it bites

One page load is about 33 calls, so roughly 4 page loads in a minute reach the limit. While PERF-01 is unfixed, 4 tab clicks do exactly that.

Measured on production

Rapid repeated loads returned this sequence: the number of generations missing their manifest rose from 0 to 8 to 16 as the limit was crossed, and the payload shrank from 403 KB to 268 KB.

The same shrinkage showed up in the 8-click baseline (the last two rows were smaller).

Hypothesis, not verified

Because every frontend server call shares the same egress address, the 120 per minute budget may be shared across all users of the deployment. This needs a second account, or the backend request logs, to confirm.

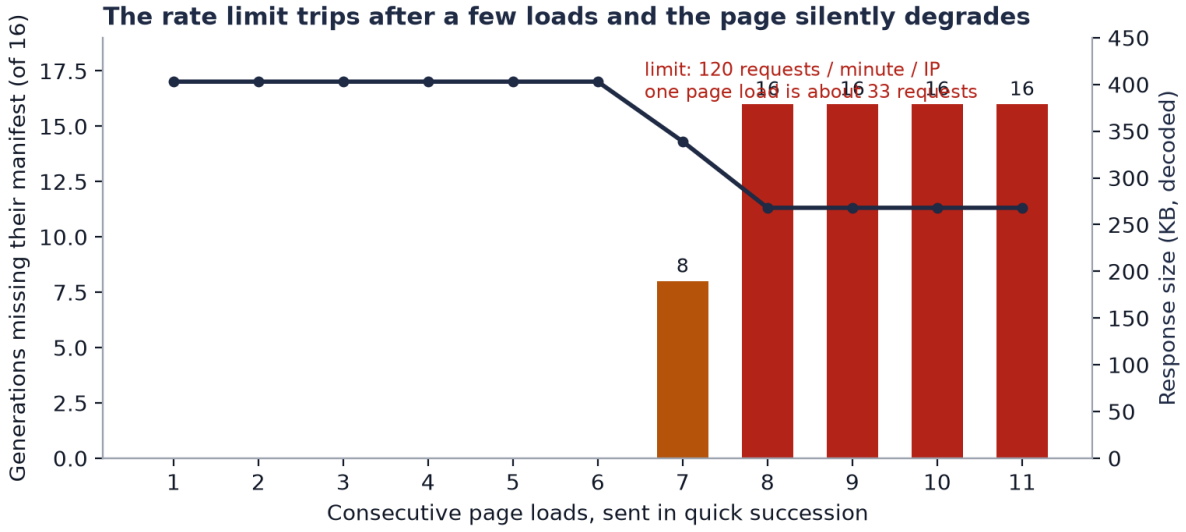


Figure 5. Generations missing their manifest, and the response size, over consecutive page loads.

Y The solution and why it is the most optimised

1. **Reduce the traffic first** with PERF-01 and PERF-02. This removes most of the load that triggers the limit.
2. **Then key the throttle per API key.** Use a custom `getTracker` based on the bearer token's key id instead of the IP, and set `trust proxy` correctly if requests pass through a proxy.

Why this is the most optimised fix

1. Raising the limit would only move the cliff and weaken protection against abuse.
2. Fewer calls fixes the cause. A per-key tracker fixes the unfairness: one user cannot lock out the rest, and a shared server address no longer means a shared budget.
3. The auth routes keep their own tighter limit (10 per minute), so brute-force protection is unaffected.

Alternatives considered and rejected

Alternative	Why not
Raise the limit from 120 to 600	Hides the real problem and weakens abuse protection.
Exempt the frontend by IP	Brittle with serverless egress addresses, and an attacker who reaches that path bypasses all limits.
Retry on 429 in the frontend	Adds latency and more load to a backend that is already limiting.

> **Decision for the owner**
> The throttle change is a backend behaviour change, so it is outside the frontend-only scope and needs explicit approval.

Z The result: expected versus actual

The Expected column is the hypothesis, written before any code. The Actual column was filled in on 30 October 2026 by re-running the same method on production after each deploy (commits `379c83c` and `1b1eb67`). Anything not re-tested on production says so.

Measure	Baseline (measured)	Expected result (hypothesis)	Actual result (production, after deploy)
Manifests missing after 10 rapid tab clicks	16 of 16 by click 5	0 of 16 (after PERF-01 the clicks make no requests)	0 of 16 after 31 rapid clicks; the Reproduction tab showed 16 partly reproducible and 0 Unknown. Met.
Page loads per minute before throttling	about 4	about 40 or more (after PERF-02 step 2)	About 40 (computed: 120 per minute per key, 3 requests per load; it was about 6 at 18 calls). Met. It rarely matters now, since tab clicks make no requests.
One user's browsing affecting another	unknown (hypothesis)	none (after the per-key tracker)	Local build: another user sending 200 requests had no effect on this user, while the old build signed this user out. Not provoked on production, because that would be a stress test of the live site.

PERF-04 Failed manifest loads are swallowed and look like "no data"

CLASSIFICATION	IMPACT	FIX EFFORT	CONFIDENCE
Reliability	High	Small	High

X The problem encountered

What happens in the code

In `page.tsx` each manifest is loaded with `getReproductionManifest(...).catch(() => null)`. Every failure becomes `null`: a 429 from the rate limit, a timeout, a 500, and a genuine "no manifest" all look identical.

What the user sees

`ReproductionTab` treats `manifest === null` as "unavailable" and computes the reproducibility grade as `unknown`. When PERF-03 triggers, the tab silently degrades at random: the same experiment shows "82% recorded" on one visit and "unavailable" on the next, with no hint that anything failed.

Why it matters beyond this one bug

It undermines a product whose purpose is reproducibility. An error that looks like missing data is worse than a visible error, because nobody knows to retry.

Y The solution and why it is the most optimised

Return an explicit per-manifest result instead of `null`: loaded, missing (a real 404) or failed (with a retryable flag). Then in `ReproductionTab` and the grade logic:

- Show "couldn't load this manifest, retry" for the failed state, with a retry that re-fetches only the failed ones.
- Treat a 404 as "no manifest recorded", as today.
- Count failed manifests separately in the summary instead of folding them into "unknown".

Why this is the most optimised fix

1. It fixes the meaning of the data, not just the symptom: the type system now forces every consumer to handle "failed".
2. It is small and local, changes no API, and stays correct after PERF-03 is fixed: any future failure (timeout, deploy blip) is handled the same way.

Alternative rejected: retrying silently on failure. It would hide the failure again and add load while the backend is limiting.

Z The result: expected versus actual

The Expected column is the hypothesis, written before any code. The Actual column was filled in on 30 October 2026 by re-running the same method on production after each deploy (commits `379c83c` and `1b1eb67`). Anything not re-tested on production says so.

Measure	Baseline (measured)	Expected result (hypothesis)	Actual result (production, after deploy)
A throttled manifest fetch is shown as	"unavailable" or "unknown", no error	"couldn't load, retry"	Local build with 8 of 16 manifests forced to 429: a banner, per-row "Rate limited" chips and a Retry button; Retry loaded all 16. Not triggered on production, because nothing was throttled there. Met locally.
Wrong "unknown" grades under load	up to 16 of 16	0 (failures are labelled, not graded)	0 on production after the 31-click burst (nothing was throttled, so the failure path was not exercised there). Failure grading verified locally.
A real "no manifest" (404)	unavailable	unchanged	Covered by unit tests; not re-tested on production.

PERF-05 The page props carry duplicated and unneeded data

CLASSIFICATION	IMPACT	FIX EFFORT	CONFIDENCE
Data design	Low-Medium	Small-Medium	High (facts)

X The problem encountered

What happens in the code

`ExperimentWorkspace` is a client component, so every prop the server page passes is serialized into the response. In the measured payload (about 275 KB decoded with manifests missing, 403 KB with them) the pieces were:

Part	Size	Used by
`trainingRuns` **prop**	about 125 KB (about 29 KB metrics and 9.5 KB health per run)	Metrics and Runs tabs, run inspector
`generations` **prop**	about 54 KB	Overview, Lineage, Reproduction, generation inspector
Python source text rows	30 copies, about 2.6 KB each (about 77 KB, 28% of the payload)	One parameter shown in the inspector

Where the Python source comes from

The SDK's `extract_domain_randomize_fn` (`integrations/mujoco_playground.py`) stores the function source in `parameters.domain_randomize.source`. Each generation carries it in `generation.parameters` and again in `generation.normalized.parameters` (the backend's additive normalized copy). That is 15 generations x 2 = 30 copies. When manifests load, the count rises to 60 (the extra 30 were not path-verified).

It is not repeated rendering

The copies are separate JSON objects, and React Server Components write each long string once as its own row and reference it. The repeated function is repeated **data**, not repeated component work.

It is not the cause of the delay

Compressed, the payload is 25 KB, because brotli removes the repetition, and the body arrives in a single burst at the end. The delay is server wait, not transfer.

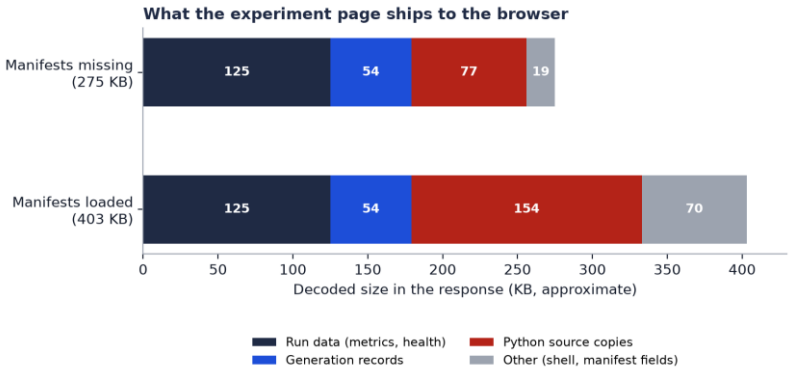


Figure 6. What the page ships to the browser, with and without manifests.

Y The solution and why it is the most optimised

1. **Drop fields the UI never reads**, in the frontend mapper before the data reaches the client component: `generation.normalized` and `manifest.generation.parameters`. First confirm by search that no component reads them.
2. **Load per-step metrics and health on demand**, when a run is selected or the Metrics tab opens. This overlaps with PERF-02 step 1.
3. **Show long strings such as the source behind a "view source" control** that fetches on demand, instead of shipping 30 copies up front.

Why this is the most optimised fix

It removes data by changing what is sent, which helps first load, memory use and parsing, not just the wire. It needs no backend change: the frontend can strip fields it does not use, and the backend response stays additive.

Alternatives rejected: compressing harder (already brotli), and moving to client-side fetching (it moves the same bytes into the browser).

Z The result: expected versus actual

The Expected column is the hypothesis, written before any code. The Actual column was filled in on 30 October 2026 by re-running the same method on production after each deploy (commits `379c83c` and `1b1eb67`). Anything not re-tested on production says so.

Measure	Baseline (measured)	Expected result (hypothesis)	Actual result (production, after deploy)
Decoded payload (manifests missing)	275 KB	about 200 KB (about 77 KB of source removed)	Not measured on production (no failure to compare against).
Decoded payload (manifests loaded)	403 KB	about 250 KB (about 150 KB of duplicated source removed)	274 KB, 32 percent smaller. The normalized mirrors are gone (16 to 0) but the source text is not, so it lands between the expected 250 KB and the old 403 KB. Partly met.
Source copies in the response	30 to 60	0 to 1	30 (was 60 with manifests loaded): the `domain_randomize` source still appears once in each generation and once in each manifest. Not met.
Compressed size	about 25 KB	little change (compression already hides the repetition)	Not re-measured.

> **What to learn from the gap**
>
> Expect the decoded size to drop clearly and the compressed size to barely move. If load time does not improve either, that confirms size was never the bottleneck.

PERF-06 Revisiting a tab reuses nothing

CLASSIFICATION	IMPACT	FIX EFFORT	CONFIDENCE
Caching and state	High	None beyond PERF-01	High

X The problem encountered

What the user sees

Open Lineage, go to Runs, return to Lineage: the return takes about as long as the first visit (Lineage: 2060 ms, then 1426 ms, then 1048 ms; the later ones are only faster because rate limiting had started).

Why: four caches exist and none applies

Layer	State here	Why it does not help
Browser HTTP cache	Response is `private, no-cache, no-store`	Intentionally uncacheable.
Next.js server data cache	Every backend call uses `cache: "no-store"` (`shared/http/http.ts:34`)	Deliberate. Prevents one user's data being served to another.
Next.js client Router Cache	`staleTimes.dynamic` defaults to 0 s, and this page reads cookies so it is dynamic	A `router.push` to a visited URL is a fresh fetch.
React state or app cache	Tab bodies unmount when inactive. Data lives in props	The router replaces the props on every navigation.

Safe to reuse	Must stay fresh
The experiment, generations, runs, metrics and evaluations already in the client's props, for the life of the page view.	Status and metrics for runs that are still training (the 5-second poll covers this), and everything after a user action.

Y The solution and why it is the most optimised

No separate change. PERF-01 removes the request, so the data already in memory is reused on every revisit. Freshness is covered by the live poll for running runs plus the explicit Refresh action.

> **Why this is the most optimised fix**
>
> Adding a cache would introduce staleness and user-isolation risk to solve a problem that disappears once the unnecessary request is removed. **Do not add caching here.**

If tab-local state should survive a switch (for example a Metrics chart's selected series), that is a separate small change: lift that state into `ExperimentWorkspace`, or keep inactive tabs mounted but hidden.

Z The result: expected versus actual

The Expected column is the hypothesis, written before any code. The Actual column was filled in on 30 October 2026 by re-running the same method on production after each deploy (commits `379c83c` and `1b1eb67`). Anything not re-tested on production says so.

Measure	Baseline (measured)	Expected result (hypothesis)	Actual result (production, after deploy)
Revisit time	1.0 to 1.4 s, same as a first visit	under 100 ms	12 and 14 ms on the two Lineage revisits. Met.
Chart selection survives a tab round trip	no	no, unless the state is lifted (decide separately)	Not changed and not re-tested; still a separate decision.

PERF-07 The workspace cannot tell us where its time goes

CLASSIFICATION	IMPACT	FIX EFFORT	CONFIDENCE
Observability	Medium	Small	High

X The problem encountered

Nothing records how long each backend call inside `page.tsx` takes. The conclusion in PERF-02 (three sequential hops of about 0.4 to 0.5 s each) is inferred from outside, using the `/live` route as a proxy. That is a hypothesis, and so are all of PERF-02's expected numbers.

Y The solution and why it is the most optimised

Add per-call timing around `backendFetch` and emit it as a `Server-Timing` response header or a structured log line (path, status, milliseconds). `Server-Timing` entries are visible in the DevTools Network panel without extra tooling.

Why this is the most optimised fix

A few lines in one function (`shared/http/http.ts`) give exact numbers for every future performance question, and turn PERF-02 from inferred into measured. Never log tokens or request bodies.

Z The result: expected versus actual

The Expected column is the hypothesis, written before any code. The Actual column was filled in on 30 October 2026 by re-running the same method on production after each deploy (commits `379c83c` and `1b1eb67`). Anything not re-tested on production says so.

Measure	Baseline (measured)	Expected result (hypothesis)	Actual result (production, after deploy)
Per-call timing visible in DevTools	none	every backend call listed with its duration	Done, but not yet read on production. The workspace route sends a `Server-Timing` header listing each part (experiment, correlation, runs, health, drift, metrics and evaluations, generations, manifests, total); the page logs it in `experiment_workspace.loaded`, and `BACKEND_FETCH_TIMING=1` logs every backend call. These are in the hosting provider's logs, not visible from the browser. Met in code, untested on production.
PERF-02's 3 hops at about 0.4 to 0.5 s	inferred	confirmed or corrected	Supported by two production measurements rather than the log: calls fell 33 to 18 with no speed-up, then removing a hop cut the server render from 2.0 s to 0.87 s, which is what the hop model predicts. Locally, the server-side part of the workspace request took 11 to 14 ms, so the time is round trips. The per-part numbers from production are still to be read.

PERF-08 The tab animation adds a fixed wait

CLASSIFICATION	IMPACT	FIX EFFORT	CONFIDENCE
Frontend rendering	Low	Trivial	High

X The problem encountered

`ExperimentWorkspace` wraps the canvas in `AnimatePresence mode="wait"` with a 0.2 s exit and a 0.2 s enter, so a tab change costs about 0.4 s even when the data is instant. Today that is hidden behind the 1 to 2 s network wait. After PERF-01 it becomes the slowest part of a tab change.

Y The solution and why it is the most optimised

Keep the animation but shorten it (about 0.12 s each way), or drop the exit phase. `useReducedMotion` is already wired, so reduced-motion users are unaffected.

Why this is the most optimised fix

It keeps the polish the product already has and removes only the dead time. It is a one-line change, and should be done after PERF-01 so the effect can actually be seen.

Z The result: expected versus actual

The Expected column is the hypothesis, written before any code. The Actual column was filled in on 30 October 2026 by re-running the same method on production after each deploy (commits `379c83c` and `1b1eb67`). Anything not re-tested on production says so.

Measure	Baseline (measured)	Expected result (hypothesis)	Actual result (production, after deploy)
Animation delay per tab change	about 0.4 s (hidden by the network)	about 0.1 to 0.25 s	No exit phase and a 0.12 s fade-in. The new tab is in the page 8 to 24 ms after the click and fades in from there. Met.

Suggested order of work

Step	Issue	Why this position
1	PERF-07 Timing	Cheap, and it gives measured numbers for everything after it.
2	PERF-01 Shallow URL updates	Biggest user-visible win, and it removes most of PERF-03's traffic.
3	PERF-04 Visible failures	A correctness fix; it should land before the throttle change so failures are never hidden.
4	PERF-08 Animation	Only visible once PERF-01 is in.
5	PERF-02 Fewer calls	A larger change; it benefits first load and refresh.
6	PERF-05 Payload	A tidy-up; do it alongside PERF-02.
7	PERF-03 Per-key throttle	A backend change that needs a decision, and it is less urgent once traffic drops.

PERF-06 needs no work of its own.

Open questions

- Is the throttle budget shared across users? Answered on a local build: yes, the old per-IP limit let one user's burst sign another out. Not provoked on production. Fixed by the per-user limit.
- Does anything read `generation.normalized` or `manifest.generation.parameters`? Searched: `normalized` is not read and was removed; `manifest.generation.parameters` is still sent because the raw-manifest view uses it.
- Should chart and table selections survive a tab switch? Still open (a product decision for PERF-06).
- Should a tab click ever re-fetch? Decided: no. A "Refresh data" button reloads on demand (works on production, about 2 s).
- Should health and drift be batched, then the whole workspace loaded in one endpoint? Done (commit `1b1eb67`): the server render fell from 2015 to 871 ms on production.
- New: should the session check and the workspace request run together? It would remove the last extra round trip (about 380 ms to about 230 ms on the local setup), but it changes the sign-in flow, so it needs a decision.

Results log

One line per measurement on production, taken after the fix was deployed. Screenshots and raw numbers are in `docs/perf-evidence/after-prod/`.

Date	Issue	Commit / PR	What was measured	Result	Matched hypothesis?
30 Oct 2026	PERF-01, 06, 08	379c83c	8-click tab sequence, UI-changed time and requests	8 to 24 ms, median 13 ms, 0 requests (was median 1922 ms)	**Yes, better**
30 Oct 2026	PERF-01, 03, 04	379c83c	30 clicks 40 ms apart, then the Reproduction tab	0 new requests, 16 of 16 manifests, 0 Unknown, still signed in	**Yes**
30 Oct 2026	PERF-01	379c83c	Back, forward and a deep link on a fresh load	URL and tab followed in each case	**Yes**

Date	Issue	Commit / PR	What was measured	Result	Matched hypothesis?
30 Oct 2026	PERF-05	379c83c	Decoded RSC payload with manifests loaded	274 KB (was 403 KB); normalized mirrors 16 to 0; source copies 30	Partly
30 Oct 2026	PERF-02	379c83c	Full page load time (RSC fetch, page ready)	2015 ms and 2439 ms; no improvement	No
30 Oct 2026	PERF-01	379c83c	Refresh data button	Shows Refreshing, finishes in about 2 s	Yes
30 Oct 2026	PERF-02	1b1eb67	Server render of the page, 5 fetches	834 to 1073 ms, median 871 ms (was 2015 ms)	Mostly (0.5 to 0.8 s expected)
30 Oct 2026	PERF-02	1b1eb67	Hard page load, 3 loads	1117 to 1994 ms, median 1604 ms (was 2439 ms)	Yes, better
30 Oct 2026	PERF-01, 02	1b1eb67	Refresh data; 8 tab clicks; 30-click burst	1257 ms (was 2074); clicks 11 to 24 ms, 0 data requests; 0 Unknown	Yes
30 Oct 2026	PERF-02	1b1eb67	Real backend and Postgres, 150 ms simulated round trip	18 calls and 3 hops, about 660 ms, down to 3 calls and 2 hops, about 380 ms; identical page	Yes